

```

// FreightFrenzyAutonomous - Nikhil
// this one's a simplified take on FREIGHT FRENZY, the 21/22 ftc season game.
// in the real game, robots read a randomised "barcode" at the start of auto to
// figure out which level of the shipping hub scores a bonus, then drive over and
// place a piece of freight on that level, before optionally spinning a carousel
// to deliver a duck at the end.
// obviously don't have the real barcode/hub here, so i'm using a colour sensor
// to stand in for "reading" the randomised element, then branching to drive to
// one of three positions depending on what it reads - same idea as picking a
// shipping hub level based on what auto decoded. same building blocks as my
// other files (encoder driving, sensor branching)

public class FreightFrenzyAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    ColorSensor color1;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        color1 = hardwareMap.get(ColorSensor.class, "color1");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);
        motorLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        motorRight.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        waitForStart();

        // "read the barcode" - standing in for the randomised level marker
        telemetry.addData("Red Value", color1.red());
        telemetry.addData("Blue Value", color1.blue());
        telemetry.update();

        // decide which shipping hub level to go for based on what we read,
        // same logic shape as real freight frenzy auto code deciding level 1/2/3
        if (color1.red() > 220) {
            // level 3 - drive furthest, most points in the real game
            moveTo(1500, 1500, 0.5);
        } else if (color1.blue() > 220) {
            // level 2 - medium distance
            moveTo(1000, 1000, 0.5);
        } else {
            // level 1 - default/fallback, closest
            moveTo(500, 500, 0.5);
        }

        // "deposit freight" - representing placing on the hub, just a short pause
        // here since i don't have an actual claw/arm mechanism built for this
        sleep(300);

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }

    private void moveTo(int leftTicks, int rightTicks, double power) {
        int leftTarget = motorLeft.getCurrentPosition() + leftTicks;
        int rightTarget = motorRight.getCurrentPosition() + rightTicks;

        motorLeft.setTargetPosition(leftTarget);
        motorRight.setTargetPosition(rightTarget);

        motorLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
    }
}

```

```
motorRight.setMode(DcMotor.RunMode.RUN_TO_POSITION);

motorLeft.setPower(power);
motorRight.setPower(power);

while (opModeIsActive() && motorLeft.isBusy() && motorRight.isBusy()) {
    telemetry.addData("Left ticks", motorLeft.getCurrentPosition());
    telemetry.addData("Right ticks", motorRight.getCurrentPosition());
    telemetry.update();
}

motorLeft.setPower(0);
motorRight.setPower(0);
}
```